policy gradient 策略梯度

基于策略的方法是显式得学习一个目标策略
首先要将策略参数化：
假设策略π_θ是一个随机策略，并且处处可微，我们可以用一个线性模型或者神经网络模型来为这个策略函数建模。（输入某个状态s，输出一个动作的概率分布）
目标函数：
寻找一个最优策略并最大化这个策略在环境中的期望回报
$J(\theta) = E_{s_0}[V^{\pi_\theta}(s_0)]$ s_0是初始状态
将目标函数对策略θ求导，使用策略梯度上升方法更新 θ ，使得 $J(\theta)$ 增大。设当前策略网络的参数为 θ_{now} ，做梯度上升更新参数，得到新的参数

$$\theta_{new} : \theta_{new} \leftarrow \theta_{now} + \beta \cdot \nabla_\theta J(\theta_{now})$$

此处的 β 是学习率，需要手动调。
其中的梯度： $\nabla_\theta J(\theta_{now}) = \frac{\partial J(\theta)}{\partial \theta} |_{\theta=\theta_{now}}$ 称为**策略梯度(policy gradient)**。

$$\begin{aligned}
\nabla_\theta J(\theta) &\propto \sum_{s \in S} \nu^{\pi_\theta}(s) \sum_{a \in A} Q^{\pi_\theta}(s,a) \nabla_\theta \pi_\theta(a|s) \\
&= \sum_{s \in S} \nu^{\pi_\theta}(s) \sum_{a \in A} \pi_\theta(a|s) Q^{\pi_\theta}(s,a) \frac{\nabla_\theta \pi_\theta(a|s)}{\pi_\theta(a|s)} \\
&= \mathbb{E}_{\pi_\theta}[Q^{\pi_\theta}(s,a) \nabla_\theta \log \pi_\theta(a|s)]
\end{aligned}$$

这个梯度可以用来更新策略。需要注意的是，因为上式中期望的下标是π_θ，所以策略梯度算法为**在线策略（on-policy）算法**，即必须使用当前策略采样得到的数据来计算梯度。直观理解一下策略梯度这个公式，可以发现在每一个状态下，梯度的修改是让策略更多地去采样到带来较高值的动作，更少地去采样到带来较低值的动作

REINFORCE算法

采用蒙特卡洛方法估计$Q^{\pi_\theta}(s,a)$，对于一个有限步数的环境来说，REINFORCE算法中的策略梯度为

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T}\left(\sum_{t'=t}^{T}\gamma^{t'-t} r_{t'}\right)\nabla_\theta \log \pi_\theta(a_t|s_t)\right]$$

T是和环境交互的最大步数

算法具体流程

- 初始化策略参数θ
- **for** 序列 $e = 1 \to E$ **do**：
- 用当前策略π_θ采样轨迹 $\{s_1, a_1, r_1, s_2, a_2, r_2, \ldots s_T, a_T, r_T\}$
- 计算当前轨迹每个时刻t往后的回报$\sum_{t'=t}^{T} \gamma^{t'-t} r_{t'}$，记为$\psi_t$
- 对θ进行更新，$\theta = \theta + \alpha \sum_t^T \psi_t \nabla_\theta \log \pi_\theta(a_t|s_t)$
- **end for**

Actor-Critic算法

既学习价值函数，又学习策略函数
目标是优化一个带参数大策略，但是会额外学习价值函数

梯度：

$$g = \mathbb{E}\left[\sum_{t=0}^{T} \psi_t \nabla_\theta \log \pi_\theta(a_t|s_t)\right]$$

其中，ψ_t可以有很多种形式：

$$\begin{aligned}
&1.\ \sum_{t'=0}^{T} \gamma^{t'} r_{t'}：轨迹的总回报；\\
&2.\ \sum_{t'=t}^{T} \gamma^{t'-t} r_{t'}：动作a_t之后的回报；\\
&3.\ \sum_{t'=t}^{T} \gamma^{t'-t} r_{t'} - b(s_t)：基准线版本的改进；\\
&4.\ Q^{\pi_\theta}(s_t, a_t)：动作价值函数；\\
&5.\ A^{\pi_\theta}(s_t, a_t)：优势函数；\\
&6.\ r_t + \gamma V^{\pi_\theta}(s_{t+1}) - V^{\pi_\theta}(s_t)：时序差分残差。
\end{aligned}$$

下面着重介绍通过时序差分残差来指导策略梯度学习
由于Actor-Critic算法可以在每一步之后进行更新，因此不对任务步数做限制。

将Actor-Critic算法分为两个部分：Actor（策略网络）和Critic（价值网络）

- Actor 要做的是与环境交互，并在 Critic 价值函数的指导下用策略梯度学习一个更好的策略。
- Critic 要做的是通过 Actor 与环境交互收集的数据学习一个价值函数，这个价值函数会用于判断在当前状态什么动作是好的，什么动作不是好的，进而帮助 Actor 进行策略更新。

Actor 的更新采用策略梯度的原则
Critic采用时序差分残差的方式：
价值网络表示为V_ω，参数为ω。
定义价值函数的损失函数：

$$\nabla_\omega \mathcal{L}(\omega) = -(r + \gamma V_\omega(s_{t+1}) - V_\omega(s_t)) \nabla_\omega V_\omega(s_t)$$

算法流程：

- 初始化策略网络参数θ，价值网络参数ω
- **for** 序列 $e = 1 \rightarrow E$ **do**：
- 用当前策略π_θ采样轨迹$\{s_1, a_1, r_1, s_2, a_2, r_2, \ldots\}$
- 为每一步数据计算: $\delta_t = r_t + \gamma V_\omega(s_{t+1}) - V_\omega(s_t)$
- 更新价值参数$w = w + \alpha_\omega \sum_t \delta_t \nabla_\omega V_\omega(s_t)$
- 更新策略参数$\theta = \theta + \alpha_\theta \sum_t \delta_t \nabla_\theta \log \pi_\theta(a_t|s_t)$
- **end for**

TRPO算法

PPO算法

PPO-Penalty

PPO-惩罚（PPO-Penalty）用拉格朗日乘数法直接将 KL 散度的限制放进了目标函数中，这就变成了一个无约束的优化问题，在迭代的过程中不断更新 KL 散度前的系数。即：

$$\arg\max_\theta \mathbb{E}_{s\sim\nu^{\pi_{\theta_k}}} \mathbb{E}_{a\sim\pi_{\theta_k}(\cdot|s)} \left[\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)} A^{\pi_{\theta_k}}(s,a) - \beta D_{KL}[\pi_{\theta_k}(\cdot|s), \pi_\theta(\cdot|s)] \right]$$

令 $d_k = D_{KL}^{\nu^{\pi_{\theta_k}}}(\pi_{\theta_k}, \pi_\theta)$，$\beta$ 的更新规则如下：

1. 如果$d_k < \delta/1.5$，那么$\beta_{k+1} = \beta_k/2$
2. 如果$d_k > \delta \times 1.5$，那么$\beta_{k+1} = \beta_k \times 2$
3. 否则$\beta_{k+1} = \beta_k$

其中，δ是事先设定的一个超参数，用于限制学习策略和之前一轮策略的差距。

PPO-Clip

PPO 的另一种形式 PPO-截断（PPO-Clip）更加直接，它在目标函数中进行限制，以保证新的参数和旧的参数的差距不会太大，即：

$$\arg\max_{\theta}\mathbb{E}_{s\sim\nu^{\pi_{\theta_k}}}\mathbb{E}_{a\sim\pi_{\theta_k}(\cdot|s)}\left[\min\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)}A^{\pi_{\theta_k}}(s,a),\text{clip}\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)},1-\epsilon,1+\epsilon\right)A^{\pi_{\theta_k}}(s,a)\right)\right]$$

其中 $\text{clip}(x,l,r):=\max(\min(x,r),l)$，即把 x 限制在 $[l,r]$ 内。上式中ϵ是一个超参数，表示进行截断（clip）的范围。

如果$A^{\pi_{\theta_k}}(s,a)>0$，说明这个动作的价值高于平均，最大化这个式子会增大$\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)}$，但不会让其超过$1+\epsilon$。反之，如果$A^{\pi_{\theta_k}}(s,a)<0$，最大化这个式子会减小$\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)}$，但不会让其超过$1-\epsilon$。如图 12-1 所示。

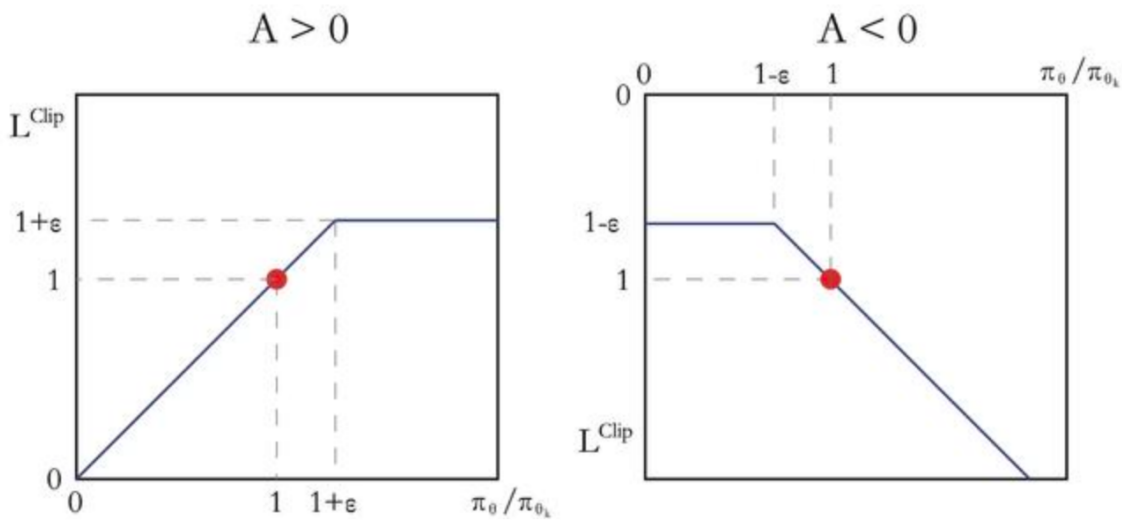

图12-1 PPO-截断示意图

代码实现

PYTHON

```python
import gym
import torch
import torch.nn.functional as F
import numpy as np
import matplotlib.pyplot as plt
import rl_utils


class PolicyNet(torch.nn.Module):
    def __init__(self, state_dim, hidden_dim, action_dim):
        super(PolicyNet, self).__init__()
        self.fc1 = torch.nn.Linear(state_dim, hidden_dim)
        self.fc2 = torch.nn.Linear(hidden_dim, action_dim)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        return F.softmax(self.fc2(x), dim=1)


class ValueNet(torch.nn.Module):
    def __init__(self, state_dim, hidden_dim):
        super(ValueNet, self).__init__()
        self.fc1 = torch.nn.Linear(state_dim, hidden_dim)
        self.fc2 = torch.nn.Linear(hidden_dim, 1)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        return self.fc2(x)


class PPO:
    ''' PPO算法,采用截断方式 '''
    def __init__(self, state_dim, hidden_dim, action_dim, actor_lr, critic_lr,
                 lmbda, epochs, eps, gamma, device):
        self.actor = PolicyNet(state_dim, hidden_dim, action_dim).to(device)
        self.critic = ValueNet(state_dim, hidden_dim).to(device)
        self.actor_optimizer = torch.optim.Adam(self.actor.parameters(),
                                                lr=actor_lr)
        self.critic_optimizer = torch.optim.Adam(self.critic.parameters(),
                                                 lr=critic_lr)
        self.gamma = gamma
        self.lmbda = lmbda
        self.epochs = epochs  # 一条序列的数据用来训练轮数
        self.eps = eps  # PPO中截断范围的参数
        self.device = device

    def take_action(self, state):
```

```python
        state = torch.tensor([state], dtype=torch.float).to(self.device)
        probs = self.actor(state)
        action_dist = torch.distributions.Categorical(probs)
        action = action_dist.sample()
        return action.item()

    def update(self, transition_dict):
        # 从传入的 transition_dict 中提取状态、动作、奖励、下一状态和终止标志信息。
        states = torch.tensor(transition_dict['states'],
                              dtype=torch.float).to(self.device)
        actions = torch.tensor(transition_dict['actions']).view(-1, 1).to(
            self.device)
        rewards = torch.tensor(transition_dict['rewards'],
                               dtype=torch.float).view(-1, 1).to(self.device)
        next_states = torch.tensor(transition_dict['next_states'],
                                   dtype=torch.float).to(self.device)
        dones = torch.tensor(transition_dict['dones'],
                             dtype=torch.float).view(-1, 1).to(self.device)
        # 计算 TD 目标（时序差分目标）
        td_target = rewards + self.gamma * self.critic(next_states) *
                    (1 -dones)
        # 计算时序差分误差
        td_delta = td_target - self.critic(states)
        # 计算优势函数 advantage，用于计算 PPO 损失函数。
        # 优势函数是 TD 误差经过处理后的版本，它可以帮助控制策略更新的幅度，减少方差。
        advantage = rl_utils.compute_advantage(self.gamma, self.lmbda,
                                               td_delta.cpu()).to(self.device)
        # 计算旧的动作对数概率（用于计算比率），并将其与计算的优势函数分离。
        old_log_probs = torch.log(self.actor(states).gather(1,
                                                            actions)).detach()

        # 进行多次（self.epochs 次）参数更新，采用 PPO 算法。
        for _ in range(self.epochs):
            # 重新计算当前策略的动作对数概率，用于计算比率。
            log_probs = torch.log(self.actor(states).gather(1, actions))
            # 计算概率比率，用于计算 PPO 损失函数的 surrogate。
            ratio = torch.exp(log_probs - old_log_probs)
            # 利用 PPO 中的截断操作，计算两个 surrogate。
            surr1 = ratio * advantage
            surr2 = torch.clamp(ratio, 1 - self.eps,
                                1 + self.eps) * advantage  # 截断
            # 选择 surrogate 中较小的值作为 Actor Loss，采用 PPO 损失函数。
            actor_loss = torch.mean(-torch.min(surr1, surr2))  # PPO损失函数
            # 使用均方误差损失函数计算 Critic Loss
            critic_loss = torch.mean(
                F.mse_loss(self.critic(states), td_target.detach()))
            # 清零梯度
            self.actor_optimizer.zero_grad()
            self.critic_optimizer.zero_grad()
            # 计算策略网络和价值网络的梯度
```

```
            actor_loss.backward()
            critic_loss.backward()
            # 更新策略网络和价值网络的参数
            self.actor_optimizer.step()
            self.critic_optimizer.step()
```